AgriScan AI – Project Overview

Made with GAMMA

1. Problem Statement

Smallholder farmers in Africa face devastating crop losses (20–40%) due to pests and diseases, yet they lack access to expert extension agents. Smartphone penetration is growing, but the **60% usage gap** (people living within network coverage who don't use mobile internet) means cloud-based AI diagnosis tools are inaccessible. Farmers need **instant, offline, and trustworthy** disease identification to take timely action.

Made with GAMMA

2. Our Solution: AgriScan AI

An **offline-first mobile app** that lets a farmer take a photo of a diseased leaf and receive:

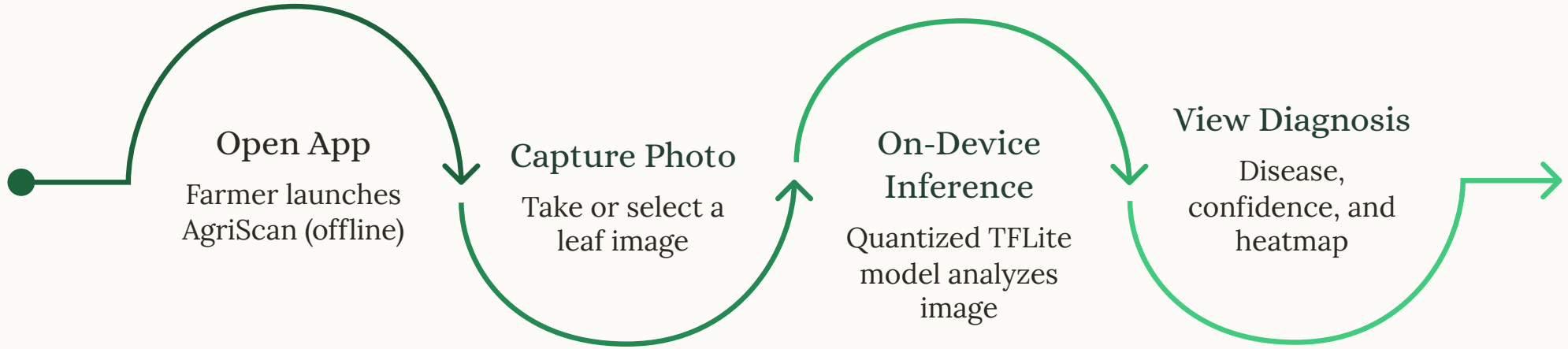
- **Instant disease diagnosis** (on-device, no internet required)
- **Explainable AI heatmap** highlighting the symptomatic regions that led to the decision – building trust and teaching the farmer what to look for.

Unique Pillars

Offline Disease Detection	Runs a lightweight CNN model entirely on the smartphone; no cloud call.	Works in remote areas with zero connectivity; solves the "usage gap".
Explainable AI (XAI)	Overlays a heatmap (Grad-CAM) on the leaf image showing which pixels influenced the diagnosis.	Builds farmer trust, enables learning, and differentiates us from black-box AI solutions. Very few hackathon projects implement XAI.

Made with GAMMA

3. How It Works (User Journey)



1. Farmer opens app (no internet needed).
2. Takes/selects a photo of a suspicious leaf.
3. App runs on-device inference using a quantized TensorFlow Lite model.
4. App displays:
   - Disease name and confidence score.
   - A heatmap overlay showing affected areas.
   - Brief treatment recommendation (pre-loaded in app).
5. If connectivity is available later, the app can sync diagnosis data for aggregated insights (optional).

Made with GAMMA

4. Technical Architecture

4.1 Mobile App (Offline-First)

- **Framework:** Flutter (cross-platform) or native Android (Kotlin) for better performance.
- **Key Features:** Camera access, image gallery, model inference, heatmap rendering, local storage of results.
- **Offline Capability:** All ML models and treatment guides bundled inside the app; no API calls needed for inference.

4.2 Machine Learning Model

- **Base Architecture:** MobileNetV3-Large (quantized INT8) for disease classification.
  - Why: Squeeze-and-Excitation modules help focus on disease lesions; excellent performance on low-end hardware.
- **Pest Detection (optional):** YOLOv8-Nano (INT8) for counting pests like Fall Armyworm.
- **Explainability:** Grad-CAM (Gradient-weighted Class Activation Mapping) to generate heatmaps.
- **Training:** TensorFlow / Keras.
- **Conversion:** TensorFlow Lite with post-training quantization (INT8) to reduce size and speed up inference.

4.3 Verified Datasets (Afrocentric)

We will use **publicly available, open-source datasets** collected in African fields:

Cassava	Cassava Spectral and Image Dataset	Harvard Dataverse	Multiple RAR archives	CMD, CBSD, Healthy
Maize	Makerere Maize	Harvard Dataverse	~9,300	MLN, MSV, Leaf Blight, Healthy
Maize (fungal)	Maize in Field (South Africa)	Kaggle	~?	Grey Leaf Spot, NCLB, Common Rust
Beans	Makerere Beans	Harvard Dataverse	~59,000	Angular Leaf Spot, Rust, Healthy
Sweet Potato	JMuBEN 3	Mendeley Data	~1,400	Leaf Rust, SPVD
Rice	Rice Leaf Disease	Mendeley Data	~10,700	Blast, Brown Spot, Bacterial Blight, Tungro
Ghana Crop Pests	Ghana Crop Pest Dataset	Mendeley Data	~24,900	Cashew, Cassava, Maize, Tomato diseases

Note: For crops like **Yam**, there is a data gap. We will use transfer learning from cassava models and supplement with web-scraped/iNaturalist images (declared in open-source disclosure).

4.4 Verified Pretrained Model Links (for transfer learning)

MobileNetV3-Large (INT8)	TFLite	Kaggle Models	Base for disease classification	**Ideal final model.** Lightweight, quantized, ready for on-device inference.
MobileNetV3-Small (INT8)	TFLite	Kaggle Models	Fallback for very low-end devices	Alternative if Large version is too slow.
YOLOv8n	PyTorch	Ultralytics	Pest detection	Train on pest data, then convert to TFLite.
Plant_Disease (InceptionResNetV2)	Keras (.h5)	Hugging Face	**Source for transfer learning**	**Do not use directly** (model is too large). Instead, use its pre-trained weights to initialize and train your lightweight MobileNetV3 model. This will improve accuracy on African crops while keeping the final app fast and offline.

Made with GAMMA

5. Possible Implementation Plan (Hackathon Timeline: 3 Mar – 3 Apr)

1: Data Curation	1–3	Download datasets, remove duplicates, augment (brightness, contrast, noise), handle class imbalance.	Cleaned dataset split into train/val/test.
2: Model Training	4–7	Train MobileNetV3-Large using transfer learning (optionally initialised from Hugging Face model). Apply post-training quantization (INT8). Validate accuracy drop <1%.	Quantized TFLite model (.tflite) + labels.
3: App + XAI	8–12	Build Flutter/Android app with camera, integrate TFLite, implement Grad-CAM (multi-output model workaround), overlay heatmap on image.	Working app that can diagnose and show heatmap.
4: Testing & Polish	13–14	Test on low-end device in airplane mode, add treatment database, record demo video.	Final prototype ready for submission.

Made with GAMMA

6. Tools & Resources

ML Development	TensorFlow, Keras	Model training
Explainable AI	tf-keras-vis	Grad-CAM implementation (Python)
Mobile Framework	Flutter + tflite_flutter or Android (Kotlin) + TensorFlow Lite	App development & on-device inference
Datasets	See Section 4.3	Training data
Pretrained Models	MobileNetV3 (ImageNet), Hugging Face Plant_Disease	Transfer learning base
Version Control	GitHub	Code repository
Design	Figma (optional)	UI mockups
Video	Loom / Screen Recorder	Demo recording

Made with GAMMA

7. Alignment with Judging Criteria

Problem-Solution Fit	Directly tackles disease detection for offline farmers – a massive real-world need.
Innovation & Creativity	Combining offline AI with explainability is novel; few hackathon projects include XAI.
Technical Implementation	Uses state-of-the-art lightweight CNNs, on-device inference, and Grad-CAM.
Scalability & Viability	App can be pre-installed on budget smartphones; model updates via Play Store; partnerships with agri-input companies.
Design & UX	Simple camera-based workflow, visual heatmap for illiterate users, multilingual text support planned.
Pitch Quality	Strong story: "AI that works where there is no internet, and shows its work."

8. Risks & Mitigation

Limited African crop diseases in training data	Use transfer learning; prioritise 5–10 most important crops for MVP (cassava, maize, beans).
Model size too large for low-end phones	INT8 quantization reduces size by 4x; target <10 MB.
Heatmap computation too slow on device	Use simplified CAM with pre-computed weights; optimise with TFLite GPU delegate if available.
Low accuracy on field images	Augment training with field-like conditions (varying light, background).
Yam data gap	Transfer learning from cassava; declare use of web-scraped images if any.

Made with GAMMA

9. Hackathon Critical Details

Registration Opens	**13 Feb 2026**
Registration & Team Formation Closes	**28 Feb 2026**
Official Kick-off	**1–2 Mar 2026**
Building Phase (Remote hacking + weekly reviews)	**3 Mar – 3 Apr 2026**
Project Submission Window	**4 Apr – 10 Apr 2026**
Grand Finale & Live Pitch	**25 Apr 2026**

Submission Requirements

- **Source Code** → GitHub public repository link
- **Pitch Deck** → Max 12 slides (PDF format)
- **Video Demo** → 3-minute maximum (YouTube unlisted or Loom)
- **Executive Summary** → One-page document
- **Working Prototype** → Mobile app with offline functionality

Judging Criteria

- Problem-Solution Fit & Real-World Impact
- Innovation & Creativity
- Technical Implementation & Functionality
- Scalability & Business Viability
- Design, UX & Accessibility
- Pitch Quality & Team Presentation

Rules & Code of Conduct

- Code must be developed **after the official kick-off** (no full pre-built projects allowed).
- Use of open-source libraries and APIs is allowed, but **must be declared**.
- All prototypes must be **safe, legal, and ethical**.
- At least **one team member must be a registered student** from a recognized African tertiary institution.
- Teams can include non-students but must not exceed **5 members**.

Made with GAMMA

10. Next Steps

01

Team sync

Confirm roles (ML, mobile, pitch).

02

Environment setup

Install TensorFlow, Flutter, clone repositories.

03

Download datasets

From links above (by 25 Feb).

04

Test baseline model

On PlantVillage to validate pipeline.

05

Prepare final overview document

For team reference.

All resources listed are open-source and compliant with hackathon rules. Code will be developed entirely after the official kick-off (3 Mar).

Made with GAMMA